

THE DSA UPLIFT

Breaking the Array Barrier

A curated 100+ problem roadmap from linear to non-linear data structures.

103

PROBLEMS

4

PHASES

100%

REFUND

01

Foundation — Arrays, Hashing, Binary Search

02

Arsenal — Sliding Window, Prefix Sums, Pointers

03

Linked Structures — LL, Stacks, Queues

04

Non-Linear — Trees, Heaps, Graphs, Tries

HOW TO USE THIS SHEET

Phase 1 & 2 — Linear Mastery

Do NOT skip these. Two Pointers, Prefix Sums, and Sliding Window are the foundational patterns required for everything that follows.

Phase 3 — Structural Shifts

Bridges the gap between contiguous memory (arrays) and pointer-based logic (linked lists, stacks). Learn how memory actually works.

Phase 4 — Non-Linear Realm

Recursion trees, graphs, heaps. Once phases 1–3 are solid, this phase clicks fast.

Implementation Note

Highly recommended to code in C++ to truly grasp memory management and pointer manipulation — though any language works fine.

DIFFICULTY KEY

● EASY

● MEDIUM

● HARD

REFUND POLICY

✓ Complete All 103 Problems

Solve every problem on this sheet and submit proof of completion. Screenshot your accepted submissions or share your LeetCode profile.

✓ 100% Money Back, No Questions Asked

If you finish the entire sheet and don't feel a meaningful improvement in your problem-solving ability, you get a full \$99 refund — immediately, no forms, no friction.

✓ How to Claim

Email your completion proof to the address on your purchase receipt within 90 days of buying. Refund is processed within 48 hours.

✓ Why We Offer This

We offer this because we know the roadmap works. The guarantee exists to remove every possible reason not to start.

01

Phase 1: The Foundation

Searching, Sorting & Frequency Maps

Master basic iteration, hash maps for frequency counting, and binary search logic. Do not skip this phase.

#	PROBLEM	CORE CONCEPT	DIFF
1	Contains Duplicate	Arrays & Hashing	Easy
2	Valid Anagram	Strings & Frequency Maps	Easy
3	Two Sum	Arrays & Hashing	Easy
4	Group Anagrams	Strings & Hashing	Medium
5	Top K Frequent Elements	Arrays & Sorting / Heaps	Medium
6	Encode and Decode Strings	String Parsing	Medium
7	Product of Array Except Self	Array Precomputation	Medium
8	Valid Sudoku	Matrix & Hashing	Medium
9	Longest Consecutive Sequence	Arrays & Hashing Set	Medium
10	Binary Search	Searching	Easy
11	Search a 2D Matrix	Matrix Binary Search	Medium
12	Koko Eating Bananas	Binary Search on Answer	Medium
13	Find Minimum in Rotated Sorted Array	Advanced Binary Search	Medium
14	Search in Rotated Sorted Array	Advanced Binary Search	Medium
15	Time Based Key-Value Store	Binary Search & Design	Medium
16	Median of Two Sorted Arrays	Binary Search	Hard
17	Merge Intervals	Sorting & Intervals	Medium
18	Insert Interval	Arrays & Intervals	Medium
19	Non-overlapping Intervals	Greedy Interval Parsing	Medium
20	Meeting Rooms	Intervals	Easy
21	Meeting Rooms II	Intervals & Min-Heap	Medium
22	Sort Colors	In-place Sorting	Medium
23	Majority Element	Moore Voting Algorithm	Easy
24	Single Number	Bit Manipulation & Arrays	Easy
25	Missing Number	Math & Arrays	Easy

02

Phase 2: The Arsenal

Prefix Sums, Sliding Window & Two Pointers

Algorithmic design patterns that reduce time complexity from $O(N^2)$ to $O(N)$. Master these and medium problems stop feeling hard.

#	PROBLEM	CORE CONCEPT	DIFF
26	Valid Palindrome	Two Pointers	Easy
27	Two Sum II — Input Array Is Sorted	Two Pointers	Medium
28	3Sum	Sorting & Two Pointers	Medium
29	Container With Most Water	Greedy Two Pointers	Medium
30	Trapping Rain Water	Hard Two Pointers	Hard
31	Best Time to Buy and Sell Stock	Sliding Window (Static)	Easy
32	Longest Substring Without Repeating Char	Sliding Window (Dynamic)	Medium
33	Longest Repeating Character Replacement	Sliding Window & Math	Medium
34	Permutation in String	Sliding Window & Hash	Medium
35	Minimum Window Substring	Hard Sliding Window	Hard
36	Sliding Window Maximum	Monotonic Deque	Hard
37	Subarray Sum Equals K	Prefix Sum & Hash Map	Medium
38	Range Sum Query — Immutable	Prefix Sum Array	Easy
39	Continuous Subarray Sum	Prefix Sum Modulo	Medium
40	Find Pivot Index	Prefix vs Suffix Sums	Easy
41	Maximum Subarray	Kadane's Algorithm	Medium
42	Maximum Product Subarray	Dynamic Kadane	Medium
43	Rotate Array	Array Reversal Technique	Medium
44	Rotate Image	Matrix Math	Medium
45	Spiral Matrix	Matrix Traversal	Medium
46	Set Matrix Zeroes	In-place Matrix State	Medium
47	Word Search	Matrix Backtracking	Medium
48	Isomorphic Strings	Dual Hash Mapping	Easy
49	Find All Anagrams in a String	Sliding Window Array	Medium
50	String to Integer (atoi)	String Parsing Edge Cases	Medium
51	Longest Palindromic Substring	Expand Around Center	Medium

03

Phase 3: Linked Structures

Nodes, Pointers, Stacks & Queues

Leave contiguous memory behind. Master how scattered memory allocations connect via pointers — the foundation for trees and graphs.

#	PROBLEM	CORE CONCEPT	DIFF
52	Reverse Linked List	Pointer Reassignment	Easy
53	Merge Two Sorted Lists	Linked List Splicing	Easy
54	Reorder List	Slow / Fast Pointer & Reverse	Medium
55	Remove Nth Node From End of List	Delayed Pointer	Medium
56	Copy List with Random Pointer	Deep Copy via Hashing	Medium
57	Add Two Numbers	Linked List Traversal Math	Medium
58	Linked List Cycle	Floyd's Cycle Detection	Easy
59	Find the Duplicate Number	Cycle Detection (Array as LL)	Medium
60	LRU Cache	Doubly Linked List + Hash Map	Medium
61	Merge k Sorted Lists	Priority Queue & LL	Hard
62	Reverse Nodes in k-Group	Advanced LL Traversal	Hard
63	Valid Parentheses	Stack Basics	Easy
64	Min Stack	Class Design & Stack Array	Medium
65	Evaluate Reverse Polish Notation	Stack Evaluation	Medium
66	Generate Parentheses	Stack & Backtracking	Medium
67	Daily Temperatures	Monotonic Stack	Medium
68	Car Fleet	Stack State Management	Medium
69	Largest Rectangle in Histogram	Hard Monotonic Stack	Hard
70	Implement Queue using Stacks	Data Structure Design	Easy
71	Implement Stack using Queues	Data Structure Design	Easy
72	Design Circular Queue	Queue Implementation	Medium
73	Number of Students Unable to Eat Lunch	Queue Simulation	Easy
74	Time Needed to Buy Tickets	Queue Optimization	Easy
75	Reveal Cards In Increasing Order	Queue Simulation / Logic	Medium
76	Asteroid Collision	Stack Mechanics	Medium

04

Phase 4: Non-Linear Realm

Trees, Heaps & Graphs

Recursive call stacks map directly to tree traversal. Once phases 1–3 are solid, this phase is where everything clicks.

#	PROBLEM	CORE CONCEPT	DIFF
77	Invert Binary Tree	Tree Recursion	Easy
78	Maximum Depth of Binary Tree	DFS Depth Traversal	Easy
79	Diameter of Binary Tree	Bottom-Up DFS	Easy
80	Balanced Binary Tree	DFS Validations	Easy
81	Same Tree	Tree Comparisons	Easy
82	Subtree of Another Tree	Nested DFS	Easy
83	Lowest Common Ancestor of a BST	BST Properties	Medium
84	Binary Tree Level Order Traversal	BFS via Queues	Medium
85	Binary Tree Right Side View	BFS / DFS Edge Traversal	Medium
86	Count Good Nodes in Binary Tree	Stateful Pre-order Traversal	Medium
87	Validate Binary Search Tree	Strict BST Bound Checking	Medium
88	Kth Smallest Element in a BST	In-order Traversal	Medium
89	Construct Binary Tree from Pre/Inorder	Array to Tree Construction	Medium
90	Binary Tree Maximum Path Sum	Hard Bottom-Up DFS	Hard
91	Serialize and Deserialize Binary Tree	Tree Parsing Strings	Hard
92	Kth Largest Element in a Stream	Min-Heap Basics	Easy
93	Last Stone Weight	Max-Heap Implementation	Easy
94	K Closest Points to Origin	Heaps with Pair Data	Medium
95	Task Scheduler	Heaps & Queues	Medium
96	Design Twitter	Heaps & OOD	Medium
97	Find Median from Data Stream	Two-Heap Architecture	Hard
98	Implement Trie (Prefix Tree)	Trie Node Construction	Medium
99	Design Add and Search Words	Trie + Backtracking	Medium
100	Word Search II	Trie + 2D Matrix DFS	Hard
101	Number of Islands	Graph BFS / DFS	Medium
102	Clone Graph	Graph Traversal & Hashing	Medium
103	Course Schedule	Topological Sort (Kahns)	Medium

AUTHOR'S NOTE

This sheet is engineered to build logic step by step — not to collect problems for the sake of it. Work through each phase in order. Build clean functions, think about time complexity, and consider memory architecture as you code. The 100% refund guarantee exists because this roadmap works. Finish every problem and feel the upgrade — or get your money back, no questions asked.